Chapter 21
Vandermonde Systems

We began, 25 years ago, to take up [the conditioning of]
the class of Vandermonde matrices.
The original motivation came from unpleasant experiences with the
computation of Gauss type quadrature rules from the
moments of the underlying weight function.

— WALTER GAUTSCHI, *How (Un)stable are Vandermonde Systems? (1990)*

Extreme ill-conditioning of the [Vandermonde] linear systems
will eventually manifest itself as n increases by yielding
an error curve which is not sufficiently levelled on the current reference . . .
or more seriously fails to have the correct number of sign changes.

— M. ALMACANY, C. B. DUNHAM, and J. WILLIAMS,
Discrete Chebyshev Approximation by Interpolating Rationals (1984)

A Vandermonde matrix is defined in terms of scalars $\alpha_0, \alpha_1, \ldots, \alpha_n \in \mathbb{C}$ by

$$V = V(\alpha_0, \alpha_1, \ldots, \alpha_n) = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ \alpha_0 & \alpha_1 & \cdots & \alpha_n \\ \vdots & \vdots & & \vdots \\ \alpha_0^n & \alpha_1^n & \cdots & \alpha_n^n \end{bmatrix} \in \mathbb{C}^{(n+1)\times(n+1)}.$$

Vandermonde matrices play an important role in various problems, such as in polynomial interpolation. Suppose we wish to find the polynomial $p_n(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_0$ that interpolates to the data $(\alpha_i, f_i)_{i=0}^n$, for distinct points α_i, that is, $p_n(\alpha_i) = f_i$, $i = 0{:}n$. Then the desired coefficient vector $a = [a_0, a_1, \ldots, a_n]^T$ is the solution of the dual Vandermonde system

$$V^T a = f \text{ (dual)}.$$

The primal system

$$Vx = b \text{ (primal)}$$

represents a moment problem, which arises, for example, when determining the weights for a quadrature rule: given moments b_i find weights x_i such that $\sum_{j=0}^n x_j \alpha_j^i = b_i$, $i = 0{:}n$.

Because a Vandermonde matrix depends on only n+ 1 parameters and has a great deal of structure, it is possible to perform standard computations with reduced complexity. The easiest algorithm to derive is for matrix inversion.

21.1. Matrix Inversion

Assume that V is nonsingular and let $V^{-1} = W = (w_{ij})_{i,j=0}^n$. The ith row of the equation $WV = I$ may be written

$$\sum_{j=0}^n w_{ij}\alpha_k^j = \delta_{ik}, \quad k = 0{:}n.$$

These equations specify a fundamental interpolation problem that is solved by the Lagrange basis polynomial:

$$\sum_{j=0}^n w_{ij}x^j = \prod_{\substack{k=0 \\ k\neq i}}^n \left(\frac{x-\alpha_k}{\alpha_i-\alpha_k}\right) =: l_i(x). \tag{21.1}$$

The inversion problem is now reduced to finding the coefficients of $l_i(x)$. It is clear from (21.1) that V is nonsingular iff the α_i are distinct. It also follows from (21. 1) that V^{-1} is given explicitly by

$$w_{ij} = \frac{(-1)^{n-j}\sigma_{n-j}(\alpha_0, \ldots, \alpha_{i-1}, \alpha_{i+1}, \ldots, \alpha_n)}{\prod_{\substack{k=0 \\ k\neq i}}^n (\alpha_i - \alpha_k)}, \tag{21.2}$$

where $\sigma_k(y_1, \ldots, y_n)$ denotes the sum of all distinct products of k of the arguments $y_1, \ldots, y_n$ (that is σ_k is the kth elementary symmetric function). An efficient way to find the w_{ij} is first to form the master polynomial

$$\phi(x) = \prod_{k=0}^{n} (x - \alpha_k) =: \sum_{i=0}^{n+1} a_i x^i,$$

and then to recover each Lagrange polynomial by synthetic division:

$$q_i(x) = \phi(x)/(x - \alpha_i),$$
$$l_i(x) = q_i(x)/q_i(\alpha_i).$$

The scalars $q_i(\alpha_i)$ can be computed by Homer's rule as the coefficients of q_i are formed.

Algorithm 21.1. Given distinct scalars $\alpha_0, \alpha_1, \ldots, \alpha_n \in \mathbb{C}$ this algorithm computes $W = (w_{ij})_{i,j=0}^{n} = V(\alpha_0, \alpha_1, \ldots, \alpha_n)^{-1}$.

```
% Stage 1: Construct the master polynomial.
a_0 = -α_0; a_1 = 1
for k = 1:n
    a_{k+1} = 1
    for j = k: -1: 1
        a_j = a_{j-1} - α_k a_j
    end
    a_0 = -α_k a_0
end

% Stage 2: Synthetic division.
for i = 0:n
    w_in = 1; s = 1
    for j = n - 1: -1:0
        w_ij = a_{j+1} + α_i w_{i,j+1}
        s = α_i s + w_ij
    end
    w(i, : ) = w(i, : )/s
end
```

Cost: $6n^2$ flops.

The $O(n^2)$ complexity is optimal, since the algorithm has n^2 output values, each of which must partake in at least one operation.

Vandermonde matrices have the deserved reputation of being ill conditioned. The ill conditioning is a consequence of the monomials being a poor

Table 21.1. *Bounds and estimates for* $\kappa_\infty(V_n)$.

	α_i	Bound or estimate	Reference
(VI):	$1/i+1$	$\kappa_\infty(V_n) > n^{n+1}$	[428, 1990]
(V2):	arbitrary	$\kappa_2(V_n) \geq n^{-1/2}2^{n-2}$	[1035, 1994]
(V3):	$\alpha_i \geq 0$	$\kappa_\infty(V_n) > 2^{n-1}$ $(n > 1)$	[429, 1988]
(V4):	equispaced [0,1]	$\kappa_\infty(V_n) \sim (4\pi)^{-1}\sqrt{2}\ 8^n$	[428, 1990]
(V5):	equispaced [– 1, 1]	$\kappa_\infty(V_n) \sim \pi^{-1}e^{-\pi/4}(3.1)^n$	[425, 1975]
(V6):	Chebyshev nodes [–1, 1]	$\kappa_\infty(V_n) \sim \frac{3^{3/4}}{4}(1+\sqrt{2})^n$	[425, 1975]
(V7):	roots of unity	$\kappa_2(V_n) = 1$	well known.

basis for the polynomials on the real line. A variety of bounds for the condition number of a Vandermonde matrix have been derived by Gautschi and his co-authors. Let $V_n = V(\alpha_0, \alpha_1, \ldots, \alpha_{n-1}) \in \mathbb{C}^{n\times n}$. For arbitrary distinct points α_i,

$$\max_i \prod_{j\neq i} \frac{\max(1, |\alpha_j|)}{|\alpha_i - \alpha_j|} \leq \|V_n^{-1}\|_\infty \leq \max_i \prod_{j\neq i} \frac{1+|\alpha_j|}{|\alpha_i - \alpha_j|}, \tag{21.3}$$

with equality on the right when $\alpha_j = |\alpha_j|e^{i\theta}$ for all j with a fixed θ (in particular, when $\alpha_j \geq 0$ for all j) [424, 1962], [426, 1978]. Note that the upper and lower bounds differ by at most a factor 2^{n-1}. More specific bounds are given in Table 21.1, on which we now comment.

Bound (V1) and estimate (V4) follow from (21.3). The condition number for the harmonic points $1/(i + 1)$ grows faster than $n!$; by contrast, the condition numbers of the notoriously ill-conditioned Hilbert and Pascal matrices grow only exponentially (see §26.1 and §26.4). For *any* choice of points the rate of growth is at least exponential (V2), and this rate is achieved for points equally spaced on [0, 1]. For points equally spaced on [– 1, 1], the condition number grows at a slower exponential rate than that for [0, 1], and the growth rate is slower still for the zeros of the nth degree Chebyshev polynomial (V6). For one set of points the Vandermonde matrix is perfectly conditioned: the roots of unity, for which $V_n/\sqrt{n}$ is unitary.

21.2. Primal and Dual Systems

The standard Vandermonde matrix can be generalized in at least two ways: by allowing confluence of the points α_i and by replacing the monomials by

other polynomials. An example of a confluent Vandermonde matrix is

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 \\ \alpha_0 & 1 & 0 & \alpha_1 & 1 \\ \alpha_0^2 & 2\alpha_0 & 2 & \alpha_1^2 & 2\alpha_1 \\ \alpha_0^3 & 3\alpha_0^2 & 6\alpha_0 & \alpha_1^3 & 3\alpha_i^2 \\ \alpha_0^4 & 4\alpha_0^3 & 12\alpha_0^2 & \alpha_1^4 & 4\alpha_i^3 \end{bmatrix}. \tag{21.4}$$

The second, third, and fifth columns are obtained by "differentiating" the previous column. The transpose of a confluent Vandermonde matrix arises in Hermite interpolation; it is nonsingular if the points corresponding to the "nonconfluent columns" are distinct.

A *Vandermonde-like matrix* is defined by $P = (p_i(\alpha_j))_{i,j=0}^{n}$, where p_i is a polynomial of degree *i*. The case of practical interest is where the pi satisfy a three-term recurrence relation. In the rest of this chapter we will assume that the p_i do satisfy a three-term recurrence relation. A particular application is the solution of certain discrete Chebyshev approximation problems [11, 1984]. Incorporating confluence, we obtain a *confluent Vandermonde-like matrix,* defined by

$$P = P(\alpha_0, \alpha_1, \ldots, \alpha_n) = [q_0(\alpha_0), q_1(\alpha_1), \ldots, q_n(\alpha_n)] \in \mathbb{C}^{(n+1)\times(n+1)},$$

where the α_i are ordered so that equal points are contiguous, that is,

$$\alpha_i = \alpha_j \quad (i < j) \quad \Rightarrow \quad \alpha_i = \alpha_{i+1} = \cdots = \alpha_j, \tag{21.5}$$

and the vectors $q_j(x)$ are defined recursively by

$$q_j(x) = \begin{cases} [p_0(x), p_1(x), \ldots, p_n(x)]^T, & \text{if } j = 0 \text{ or } \alpha_j \neq \alpha_{j-1}, \\ \dfrac{d}{dx} q_{j-1}(x), & \text{otherwise.} \end{cases}$$

For all polynomials and points, P is nonsingular; this follows from the derivation of the algorithms below. One reason for the interest in Vandermonde-like matrices is that for certain polynomials they tend to be better conditioned than Vandermonde matrices (see, for example, Problem 21.5). Gautschi [427, 1983] derives bounds for the condition numbers of Vandermonde-like matrices.

Fast algorithms for solving the confluent Vandermonde-like primal and dual systems $Px = b$ and $P^T a = f$ can be derived under the assumption that the $p_j(x)$ satisfy the three-term recurrence relation

$$p_{j+1}(x) = \theta_j(x - \beta_j)p_j(x) - \gamma_j p_{j-1}(x), \quad j \geq 1, \tag{21.6a}$$

$$p_0(x) = 1, \qquad p_1(x) = \theta_0(x - \beta_0)p_0(x), \tag{21.6b}$$

where $\theta_j \neq 0$ for all *j*. Note that in this chapter γ_i denotes a constant in the recurrence relation and not $iu/(1 - iu)$ as elsewhere in the book. The latter notation is not used in this chapter.

The algorithms exploit the connection with interpolation. Denote by $r(i) \geq 0$ the smallest integer for which $\alpha_i = \alpha_{i-1} = \cdots = \alpha_{r(i)}$. Considering first the dual system $P^T a = f$, we note that

$$\psi(x) = \sum_{i=0}^{n} a_i p_i(x) \tag{21.7}$$

satisfies

$$\psi^{(i-r(i))}(\alpha_i) = f_i, \qquad i = 0{:}n.$$

Thus ψ is a Hermite interpolating polynomial for the data $\{\alpha_i, f_i\}_0^n$, and our task is to obtain its representation in terms of the basis $\{p_i(x)\}_{i=0}^n$. As a first step we construct the divided difference form of ψ,

$$\psi(x) = \sum_{i=0}^{n} c_i \prod_{j=0}^{i-1} (x - \alpha_j). \tag{21.8}$$

The (confluent) divided differences $c_i = f[\alpha_0, \alpha_1, \ldots, \alpha_i]$ may be generated using the recurrence relation

$$f[\alpha_{j-k-1}, \ldots, \alpha_j] = \begin{cases} \dfrac{f[\alpha_{j-k}, \ldots, \alpha_j] - f[\alpha_{j-k-1}, \ldots, \alpha_{j-1}]}{\alpha_j - \alpha_{j-k-1}}, & \alpha_j \neq \alpha_{j-k-1}, \\ \dfrac{f_{r(j)+k+1}}{(k+1)!}, & \alpha_j = \alpha_{j-k-1}. \end{cases} \tag{21.9}$$

Now we need to generate the a_i in (21.7) from the c_i in (21.8). The idea is to expand (21.8) using nested multiplication and use the recurrence relations (21.6) to express the results as a linear combination of the p_j. Define

$$q_n(x) = c_n, \tag{21.10}$$

$$q_k(x) = (x - \alpha_k) q_{k+1}(x) + c_k, \quad k = n-1{:}-1{:}0, \tag{21.11}$$

from which $q_0(x) = \psi(x)$. Let

$$q_k(x) = \sum_{j=0}^{n-k} a_{k+j}^{(k)} p_j(x). \tag{21.12}$$

To obtain recurrences for the coefficients $a_j^{(k)}$ we expand the right-hand side of (21.11), giving

$$q_k(x) = (x - \alpha_k) \sum_{j=0}^{n-k-1} a_{k+j+1}^{(k+1)} p_j(x) + c_k.$$

Using the relations, from (21.6),

$$xp_0(x) = \frac{1}{\theta_0}p_1(x) + \beta_0,$$
$$xp_j(x) = \frac{1}{\theta_j}\big(p_{j+1}(x) + \gamma_j p_{j-1}(x)\big) + \beta_j p_j(x), \quad j \geq 1,$$

we obtain, for $k = 0{:}n-2$,

$$\begin{aligned} q_k(x) &= a_{k+1}^{(k+1)}\left(\frac{1}{\theta_0}p_1(x) + \beta_0\right) \\ &\quad + \sum_{j=1}^{n-k-1} a_{k+j+1}^{(k+1)}\left(\frac{1}{\theta_j}\big(p_{j+1}(x) + \gamma_j p_{j-1}(x)\big) + \beta_j p_j(x)\right) \\ &\quad - \alpha_k \sum_{j=0}^{n-k-1} a_{k+j+1}^{(k+1)} p_j(x) + c_k \\ &= c_k + (\beta_0 - \alpha_k)a_{k+1}^{(k+1)} + \frac{\gamma_1}{\theta_1}a_{k+2}^{(k+1)} \\ &\quad + \sum_{j=1}^{n-k-2}\left(\frac{1}{\theta_{j-1}}a_{k+j}^{(k+1)} + (\beta_j - \alpha_k)a_{k+j+1}^{(k+1)} + \frac{\gamma_{j+1}}{\theta_{j+1}}a_{k+j+2}^{(k+1)}\right)p_j(x) \\ &\quad + \left(\frac{1}{\theta_{n-k-2}}a_{n-1}^{(k+1)} + (\beta_{n-k-1} - \alpha_k)a_n^{(k+1)}\right)p_{n-k-1}(x) \\ &\quad + \frac{1}{\theta_{n-k-1}}a_n^{(k+1)}p_{n-k}(x), \end{aligned} \tag{21.13}$$

in which the empty summation is defined to be zero. For the special case $k = n - 1$ we have

$$q_{n-1}(x) = c_{n-1} + (\beta_0 - \alpha_{n-1})a_n^{(n)} + \frac{1}{\theta_0}a_n^{(n)}p_1(x). \tag{21.14}$$

Recurrences for the coefficients $a_j^{(k)}$, $j = k{:}n$, in terms of $a_j^{(k+1)}$, $j = k+1{:}n$, follow immediately by comparing (21.12) with (21.13) and (21.14).

In the following algorithm, stage I computes the confluent divided differences and stage II implements the recurrences derived above.

Algorithm 21.2 (dual, $P^T a = f$). Given parameters $\{\theta_j, \beta_j, \gamma_j\}_{j=0}^{n-1}$, a vector f, and points $\alpha(0{:}n) \in \mathbb{C}^{n+1}$ satisfying (21.5), this algorithm solves the dual confluent Vandermonde-like system $P^T a = f$.

```
% Stage I:
Set c = f
```

```
for k = 0:n - 1
    clast = c_k
    for j = k + 1:n
        if α_j = α_{j-k-1} then
            c_j = c_j/(k + 1)
        else
            temp = c_j
            c_j = (c_j - clast)/(α_j - α_{j-k-1})
            clast = temp
        end
    end
end

% Stage II:
Set a = c
a_{n-1} = a_{n-1} + (β_0 - α_{n-1})a_n
a_n = a_n/θ_0
for k = n - 2: -1:0
    a_k = a_k + (β_0 - α_k)a_{k+1} + (γ_1/θ_1)a_{k+2}
    for j = 1:n - k - 2
        a_{k+j} = a_{k+j}/θ_{j-1} + (β_j - α_k)a_{k+j+1} + (γ_{j+1}/θ_{j+1})a_{k+j+2}
    end
    a_{n-1} = a_{n-1}/θ_{n-k-2} + (β_{n-k-1} - α_k)a_n
    a_n = a_n/θ_{n-k-1}
end
```

Assuming that the values γ_j/θ_j are given (note that γ_j appears only in the terms γ_j/θ_j), the computational cost of Algorithm 21.2 is at most $9n^2/2$ flops. The vectors c and a have been used for clarity; in fact both can be replaced by f, so that the right-hand side is transformed into the solution without using any extra storage.

Values of θ_j, β_j, γ_j for some polynomials of interest are collected in Table 21.2.

The key to deriving a corresponding algorithm for solving the primal system is to recognize that Algorithm 21.2 implicitly computes a factorization of P^{-T} into the product of $2n$ triangular matrices. In the rest of this chapter we adopt the convention that the subscripts of all vectors and matrices run from 0 to n. In stage I, letting $c^{(k)}$ denote the vector c at the start of the kth iteration of the outer loop, we have

$$c^{(0)} = f, \qquad c^{(k+1)} = L_k c^{(k)}, \quad k = 0{:}n-1. \tag{21.15}$$

The matrix L_k is lower triangular and agrees with the identity matrix in rows

Table 21.2. *Parameters in the three-term recurrence* (21.6).

Polynomial	θ_j	β_j	γ_j	
Monomials	1	0	0	
Chebyshev	2^*	0	1	$^*\theta_0 = 1$
Legendre*	$\frac{2j+1}{j+1}$	0	$\frac{j}{j+1}$	$^*p_j(1) = 1$
Hermite	2	0	$2j$	
Laguerre	$-\frac{1}{j+1}$	$2j+1$	$\frac{j}{j+1}$	

0 to k. The remaining rows can redescribed, for $k + 1 \le j \le n$, by

$$e_j^T L_k = \begin{cases} e_j^T/(k+1), & \text{if } \alpha_j = \alpha_{j-k-1}, \\ (e_j^T - e_s^T)/(\alpha_j - \alpha_{j-k-1}), & \text{some } s < j, \text{ otherwise,} \end{cases}$$

where e_j is column j of the identity matrix. Similarly, stage II can be expressed as

$$a^{(n)} = c^{(n)}, \qquad a^{(k)} = U_k a^{(k+1)}, \quad k = n-1\colon -1\colon 0. \tag{21.16}$$

The matrix U_k is upper triangular, it agrees with the identity matrix in rows 0 to $k-1$, and it has zeros everywhere above the first two superdiagonals.

From (21.15) and (21.16) we see that the overall effect of Algorithm 21.2 is to evaluate, step-by-step, the product

$$a = U_0 \ldots U_{n-1} L_{n-1} \ldots L_0 f \equiv P^{-T} f. \tag{21.17}$$

Taking the transpose of this product we obtain a representation of P^{-1}, from which it is easy to write down an algorithm for computing $x = P^{-1}b$.

Algorithm 21.3 (primal, $Px = b$). Given parameters $\{\theta_j, \beta_j, \gamma_j\}_{j=0}^{n-1}$, a vector b, and points $\alpha(0\colon n) \in \mathbb{C}^{n+1}$ satisfying (21.5), this algorithm solves the primal confluent Vandermonde-like system $Px = b$.

```
% Stage I:
Set d = b
for k = 0:n-2
    for j = n-k:-1:2
        d_{k+j} = (γ_{j-1}/θ_{j-1}) d_{k+j-2} + (β_{j-1} - α_k) d_{k+j-1} + d_{k+j}/θ_{j-1}
    end
    d_{k+1} = (β_0 - α_k) d_k + d_{k+1}/θ_0
end
d_n = (β_0 - α_{n-1}) d_{n-1} + d_n/θ_0
```

```
% Stage II:
Set x = d
for k = n - 1: - 1:0
    xlast = 0
    for j = n: - 1 :k + 1
        if αj = αj-k-1 then
            xj = xj/(k + 1)
        else
            temp = xj/(αj - αj-k-1)
            xj = temp - xlast
            xlast = temp
        end
    end
    xk = xk - xlast
end
```

Algorithm 21.3 has, by construction, the same operation count as Algorithm 21.2.

21.3. Stability

Algorithms 21.2 and 21.3 have interesting stability properties. Depending on the problem parameters, the algorithms can range from being very stable (in either a backward or forward sense) to very unstable.

When the pa are the monomials and the points α_i are distinct, the algorithms reduce to those of Björck and Pereyra [121, 1970]. Björck and Pereyra found that for the system $Vx = b$ with $\alpha_i = 1/(i + 3)$, $b_i = 2^{-i}$, $n = 9$, and on a computer with $u \approx 10^{-16}$,

$$\kappa_\infty(V) = 9 \times 10^{13}, \quad \max_i \frac{|x_i - \hat{x}_i|}{|x_i|} = 5u.$$

Thus the computed solution has a tiny componentwise relative error, despite the extreme ill condition of V. Björck and Pereyra comment "It seems as if at least some problems connected with Vandermonde systems, which traditionally have been considered too ill-conditioned to be attacked, actually can be solved with good precision." This high accuracy can be explained with the aid of the error analysis below.

The analysis can be kept quite short by exploiting the interpretation of the algorithms in terms of matrix–vector products. Because of the inherent duality between Algorithms 21.2 and 21.3, any result for one has an analogue for the other, so we will consider only Algorithm 21.2.

The analysis that follows is based on the model (2.4), and so is valid only for machines with a guard digit. With the no-guard-digit model (2.6) the

bounds become weaker and more complicated, because of the importance of terms $fl(\alpha_j - \alpha_{j-k-1})$ in the analysis.

21.3.1. Forward Error

Theorem 21.4. *If no underflow or overflows are encountered then Algorithm* 21.2 *runs to completion and the computed solution* $\widehat{a}$ *satisfies*

$$|a - \widehat{a}| \le c(n,u)\,|U_0|\dots|U_{n-1}||L_{n-1}|\dots|L_0||f|, \tag{21.18}$$

where $c(n, u) := (1 + u)^{7n} - 1 = 7nu + 0(u^2)$.

Proof. First, note that Algorithm 21.2 must succeed in the absence of underflow and overflow, because division by zero cannot occur.

The analysis of the computation of the $c^{(k)}$ vectors is exactly the same as that for the nonconfluent divided differences in §5.3 (see (5.9) and (5.10)). However, we obtain a slightly cleaner error bound by dropping the γ_k notation and instead writing

$$\widehat{c}^{(k+1)} = (L_k + \Delta L_k)\widehat{c}^{(k)}, \qquad |\Delta L_k| \le [(1 + u)^3 - 1]|L_k|. \tag{21.19}$$

Turning to the equations (21.16), we can regard the multiplication $a^{(k)} = U_k a^{(k+1)}$ as comprising a sequence of three-term inner products. Analysing these in standard fashion we arrive at the equation

$$\widehat{a}^{(k)} = (U_k + \Delta U_k)\widehat{a}^{(k+1)}, \qquad |\Delta U_k| \le [(1 + u)^4 - 1]|U_k|, \tag{21.20}$$

where we have taken into account the rounding errors in forming $u_{i,i+1}^{(k)}$ $\beta_j - \alpha_k$ and $u_{i,i+2}^{(k)} = \gamma_{j+1}/\theta_{j+1}$ $(i = k + j)$.

Since $\widehat{c}^{(0)} = f$, and $\widehat{a} = \widehat{a}^{(0)}$, (21.19) and (21.20) imply that

$$\widehat{a} = (U_0 + \Delta U_0)\dots(U_{n-1} + \Delta U_{n-1})(L_{n-1} + \Delta L_{n-1})\dots(L_0 + \Delta L_0)f. \tag{21.21}$$

Applying Lemma 3.7 to (21.21) and using (21.17), we obtain the desired bound for the forward error. □

The product $|U_0| \dots |U_{n-1}||L_{n-1}| \dots |L_0|$ in (21.18) is an upper bound for $|U_0 \dots U_{n-1}L_{n-1} \dots L_0| = |P^T|$ and is equal to it when there is no subtractive cancellation in the latter product. To gain insight, suppose the points are distinct and consider the case $n = 3$. We have

$$\begin{aligned} P^{-T} &= U_0U_1U_2L_2L_1L_0 \\ &\equiv \begin{bmatrix} 1 & \beta_0-\alpha_0 & \gamma_1/\theta_1 & 0 \\ & \theta_0^{-1} & \beta_1-\alpha_0 & \gamma_2/\theta_2 \\ & & \theta_1^{-1} & \beta_2-\alpha_0 \\ & & & \theta_2^{-1} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ & 1 & \beta_0-\alpha_1 & \gamma_1/\theta_1 \\ & & \theta_0^{-1} & \beta_1-\alpha_1 \\ & & & \theta_1^{-1} \end{bmatrix} \end{aligned}$$

$$\times \begin{bmatrix} 1 & 0 & 0 & 0 \\ & 1 & 0 & 0 \\ & & 1 & \beta_0 - \alpha_2 \\ & & & \theta_0^{-1} \end{bmatrix} \begin{bmatrix} 1 & & & \\ 0 & 1 & & \\ 0 & 0 & 1 & \\ 0 & 0 & \frac{-1}{\alpha_3-\alpha_0} & \frac{1}{\alpha_3-\alpha_0} \end{bmatrix}$$

$$\times \begin{bmatrix} 1 & & & \\ 0 & 1 & & \\ 0 & \frac{-1}{\alpha_2-\alpha_0} & \frac{1}{\alpha_2-\alpha_0} & \\ 0 & 0 & \frac{-1}{\alpha_3-\alpha_1} & \frac{1}{\alpha_3-\alpha_1} \end{bmatrix}$$

$$\times \begin{bmatrix} 1 & & & \\ \frac{-1}{\alpha_1-\alpha_0} & \frac{1}{\alpha_1-\alpha_0} & & \\ 0 & \frac{-1}{\alpha_2-\alpha_1} & \frac{1}{\alpha_2-\alpha_1} & \\ 0 & 0 & \frac{-1}{\alpha_3-\alpha_2} & \frac{1}{\alpha_3-\alpha_2} \end{bmatrix}. \tag{21.22}$$

There is no subtractive cancellation in this product as long as each matrix has the alternating (checkerboard) sign pattern defined, for $A = (a_{ij})$, by $(-1)^{i+j}a_{ij} \geq 0$. This sign pattern holds for the matrices L_i if the points α_i are arranged in increasing order. The matrices U_i have the required sign pattern provided that (in general)

$$\theta_i > 0, \quad \gamma_i > 0 \quad \text{for all } i, \quad \text{and} \quad \beta_i - \alpha_k < 0 \text{ for all } i + k < n - 1.$$

In view of Table 21.2 we have the following result.

Corollary 21.5. *If $0 \leq \alpha_0 < \alpha_1 < \cdots < \alpha_n$ then for the monomials, or the Chebyshev, Legendre, or Hermite polynomials,*

$$|a - \hat{a}| \leq c(n,u)\,|P^{-T}|\,|f|. \qquad \square$$

Corollary 21.5 explains the high accuracy observed by Björck and Pereyra. Note that if

$$|P^{-T}|\,|f| \leq t_n |P^{-T} f| = t_n |a|$$

then, under the conditions of the corollary, $|\hat{a} - a| \leq c(n,u)t_n|a|$, which shows that the componentwise relative error is bounded by $c(n, u)t_n$. For the problem of Björck and Pereyra it can be shown that $t_n \approx n^4/24$. Another factor contributing to the high accuracy in this problem is that many of the subtractions $\alpha_j - \alpha_{j-k-1} = 1/(j + 3) - 1/(j - k + 2)$ are performed exactly, in view of Theorem 2.5.

Note that under the conditions of the corollary P-T has the alternating sign pattern, since each of its factors does. Thus if $(-1)^i f_i \geq 0$ then $t_n = 1$, and the corollary implies that $\hat{a}$ is accurate essentially to full machine precision, *independent of the condition number* $\kappa_\infty(P)$. In particular, taking f to be a column of the identity matrix shows that we can compute the inverse of P to high relative accuracy, independent of its condition number.

21.3.2. Residual

Next we look at the residual, $r = f - P^T\hat{a}$. Rewriting (21.21),

$$f = (L_0+\Delta L_0)^{-1}\dots(L_{n-1}+\Delta L_{n-1})^{-1}(U_{n-1}+\Delta U_{n-1})^{-1}\dots(U_0+\Delta U_0)^{-1}\hat{a}. \tag{21.23}$$

From the proof of Theorem 21.4 and the relation (5.9) it is easy to show that

$$(L_k + \Delta L_k)^{-1} = L_k^{-1} + E_k, \qquad |E_k| \le [(1 - u)^{-3} - 1]|L_k^{-1}|$$

Strictly, an analogous bound for $(U_k + \Delta U_k)^{-1}$ does not hold, since ΔU_k cannot be expressed in the form of a diagonal matrix times U_k. However, it seems reasonable to make a simplifying assumption that such a bound is valid, say

$$(U_k + \Delta U_k)^{-1} = U_k^{-1} + F_k, \qquad |F_k| \le [(1 - u)^{-4} - 1]|U_k^{-1}|. \tag{21.24}$$

Then, writing (21.23) as

$$f = (L_0^{-1} + E_0)\dots(L_{n-1}^{-1} + E_{n-1})(U_{n-1}^{-1} + F_{n-1})\dots(U_0^{-1} + F_0)\hat{a}$$

and invoking Lemma 3.7, we obtain the following result.

Theorem 21.6. *Under the assumption* (21.24), *the residual of the computed solution* $\hat{a}$ *from Algorithm* 21.2 *is bounded by*

$$|f - P^T\hat{a}| \le d(n,u)\,|L_0^{-1}|\dots|L_{n-1}^{-1}||U_{n-1}^{-1}|\dots|U_0^{-1}||\hat{a}|, \tag{21.25}$$

where $d(n, u) := (1 - u)^{-7n} - 1 = 7nu + O(u^2)$. □

For the monomials, with distinct, nonnegative points arranged in increasing order, the matrices L_i and U_i are bidiagonal with the alternating sign property, as we saw above. Thus $L_i^{-1} \ge 0$ and $U_i^{-1} \ge 0$, and since $P^T = L_0^{-1}\dots L_{n-1}^{-1}U_{n-1}^{-1}\dots U_0^{-1}$, we obtain from (21.25) the following pleasing result, which guarantees a tiny componentwise relative backward error.

Corollary 21.7. *Let* $0 \le \alpha_0 < \alpha_1 < \cdots < \alpha_n$, *and consider Algorithm* 21.2 *for the monomials. Under the assumption* (21.24), *the computed solution* $\hat{a}$ *satisjies*

$$|f - P^T\hat{a}| \le d(n,u)\,|P^T||\hat{a}|. \qquad \square$$

Table 21.3. *Results for dual Chebyshev-Vandermonde-like system.*

n	10	20	30	40
$\frac{\|a-\widehat{a}\|_\infty}{\|a\|_\infty}$	2.5×10^{-12}	6.3×10^{-7}	4.7×10^{-2}	1.8×10^{3}
$\frac{\|f-P^T\widehat{a}\|_\infty}{\|P\|_\infty\|\widehat{a}\|_\infty+\|f\|_\infty}$	6.0×10^{-13}	1.1×10^{-7}	5.3×10^{-3}	8.3×10^{-2}

21.3.3. Dealing with Instability

The potential instability of Algorithm 21.2 is illustrated by the following example. Take the Chebyshev polynomials T_i with the points $\alpha_i = \cos(i\pi/n)$ (the extrema of T_n), and define the right-hand side by $f_i = (-1)^i$. The exact solution to $P^T a = f$ is the last column of the identity matrix. Relative errors and residuals are shown in Table 21.3 ($u = 10^{-16}$). Even though $_{k2}(P) \leq 2$ for all n (see Problem 21.7), the forward errors and relative residuals are large and grow with n. The reason for the instability is that there are large intermediate numbers in the algorithm (at the start of stage II for n = 40, $\|c\|_\infty$ is of order 10^{15}); hence severe cancellation is necessary to produce the final answer of order 1. Looked at another way, the factorization of P^T used by the algorithm is unstable because it is very sensitive to perturbations in the factors.

How can we overcome this instability? There are two possibilites: prevention and cure. The only means at our disposal for *preventing* instability is to reorder the points α_i. The ordering is arbitrary subject to condition (21.5) being satisfied. Recall that the algorithms construct an LU factorization of P^T in factored form, and note that permuting the rows of P^T is equivalent to reordering the points α_i. A reasonable approach is therefore to take whatever ordering of the points would be produced by Gaussian elimination with partial pivoting (GEPP) applied to P^T. Since the diagonal elements of U in $P^T = LU$ have the form

$$u_{ii} = h_i \prod_{j=0}^{i-1} (\alpha_i - \alpha_j), \quad i = 0\colon n,$$

where ha depends only on the θ_i, and since partial pivoting maximizes the pivot at each stage, this ordering of the α_i can be computed at the start of the algorithm in n^2 flops (see Problem 21.8). This ordering is essentially the Leja ordering (5.13) (the difference is that partial pivoting leaves α_0 unchanged).

To attempt to *cure* observed instability we can use iterative refinement in fixed precision. Ordinarily, residual computation for linear equations is

trivial, but in this context the coefficient matrix is not given explicitly and computing the residual turns out to be conceptually almost as difficult, and computationally as expensive, as solving the linear system!

To compute the residual for the dual system we need a means for evaluating $\psi(t)$ in (21.7) and its first $k \le n$ derivatives, where $k = \max_i(i - r(i))$ is the order of confluence. Since the polynomials p_j satisfy a three-term recurrence relation we can use an extension of the Clenshaw recurrence formula (which evaluates ψ but not its derivatives). The following algorithm implements the appropriate recurrences, which are given by Smith [927, 1965] (see Problem 21.9).

Algorithm 21.8 (extended Clenshaw recurrence). This algorithm computes the $k + 1$ values $y_j = \psi^{(j)}(x)$, $0 \le j \le k$, where ψ is given by (21.7) and $k \le n$. It uses a work vector z of order k.

```
Set y(0: k) = z(0: k) = 0
y0 = an
for j = n - 1: - 1:0
    temp = y0
    y0 = θj(x - βj)y0 - γj+1z0 + aj
    z0 = temp
    for i = 1: min(k, n - j)
        temp = yi
        yi = θj((x - βj)yi + zi-1) - γj+1zi
        zi = temp
    end
end
m = 1
for i = 2:k
    m = m * i
    yi = m * yi
end
```

Computing the residual using Algorithm 21.8 costs between $3n^2$ flops (for full confluence) and $6n^2$ flops (for the nonconfluent case); recall that Algorithm 21.2 costs at most $9n^2/2$ flops!

The use of iterative refinement can be justified with the aid of Theorem 11.3. For (confluent) Vandermonde matrices, the residuals are formed using Homer's rule and (11 .7) holds in view of (5.3) and (5.7). Hence for standard Vandermonde matrices, Theorem 11.3 leads to an asymptotic componentwise backward stability result. A complete error analysis of Algorithm 21.8 is not available for (confluent) Vandermonde-like matrices, but it is easy to see that (11.7) will not always hold. Nevertheless it is clear that

a *normwise* bound can be obtained (see Oliver [805, 1977] for the special case of the Chebyshev polynomials) and hence an asymptotic normwise stability result can be deduced from Theorem 11.3. Thus there is theoretical backing for the use of iterative refinement with Algorithm 21.8.

Numerical experiments using Algorithm 21.8 in conjunction with the partial pivoting reordering and fixed precision iterative refinement show that both techniques are effective means for stabilizing the algorithms, but that iterative refinement is likely to fail once the instability y is sufficiently severe. Because of its lower cost, the reordering approach is preferable.

Two heuristics are worth noting. Consider a (confluent) Vandermonde-like system $Px = b$ *Heuristic* 1: it is systems with a large normed solution ($\|x\| \approx \|P^{-1}\|\,\|b\|$) that are solved to high accuracy by the fast algorithms. To produce a large solution the algorithms must sustain little cancellation, and the error analysis shows that cancellation is the main cause of instability. *Heuristic* 2: GEPP is unable to solve accurately Vandermonde systems with a very large normed solution ($\|x\| \gg u^{-1}\|b\|/\|P\|$). The pivots for GEPP will tend to satisfy $|u_{ii}| \gtrsim u\|P\|$, so that the computed solution will tend to satisfy $\|\widehat{x}\| \leq u^{-1}\|b\|/\|P\|$. A consequence of these two heuristics is that for Vandermonde(-like) systems with a very large-normed solution the fast algorithms will be much more accurate (but no more backward stable) than GEPP. However, we should be suspicious of any framework in which such systems arise; although the solution vector may be obtained accurately (barring overflow), subsequent computations with numbers of such a wide dynamic range will probably themselves be unstable.

21.4. Notes and References

The formulae (21. 1) and (21.2), and inversion methods based on these formulae, have been discovered independently by many authors. Traub [1013, 1966, §14] gives a short historical survey, his earliest reference being a 1932 book by Kowalewski. There does not appear to be any published error analysis for Algorithm 21.1 (see Problem 21.3). There is little justification for using the output of the algorithm to solve the primal or dual linear system, as is done in [842, 1992, §2.8]; Algorithms 21.2 and 21.3 are more efficient and almost certainly at least as stable. Calvetti and Reichel [179, 1993] generalize Algorithm 21.1 to Vandermonde-like matrices, but they do not present any error analysis. Gohberg and Olshevsky [456, 1994] give another $O(n^2)$ flops algorithm for inverting a Chebyshev–Vandermonde matrix.

The standard condition number $\kappa(V)$ is not an appropriate measure of sensitivity when only the points α_i are perturbed, because it does not reflect the special structure of the perturbations. Appropriate condition numbers were first derived by Higham [533, 1987] and are comprehensively investigated